Int. J. Fuzzy Syst. (2024) 26(7):2224–2245
https://doi.org/10.1007/s40815-024-01731-1

Quantum Finance and Fuzzy Reinforcement Learning-Based Multi-agent Trading System

Chi Cheng[1] · Bingshen Chen[1] · Ziting Xiao[1] · Raymond S. T. Lee[1]

Received: 8 January 2023 / Revised: 25 February 2024 / Accepted: 13 March 2024 / Published online: 25 May 2024

Abstract In a volatile stock market, an investor's long-term goal involves determining the most effective buying, selling strategies, and money management techniques in order to maximize profits. This paper introduces a multi-agent trading system to achieve this goal, termed QF-FRL, based on quantum finance and fuzzy reinforcement learning (QF-FRL). The system comprises two agents: (1) The trading agent, constructed using the Deep Deterministic Policy Gradient (DDPG) and Twin Delayed Deep Deterministic Policy Gradient (TD3). This agent employs a Denoising Auto Encoder (DAE) to extract stock representations from historical time series data. The trading agent initially employed the DDPG model, which was subsequently supplanted by the TD3 model. It integrates traditional financial technology indicators, like moving averages, with modern deep reinforcement learning technology to generate buying and selling signals for determining the optimal strategy. (2) The risk control agent, founded on quantum finance and an adaptive network-based fuzzy inference system. This agent merges the QPL indicator with a fuzzy risk control method to ascertain transaction amounts. Furthermore, a genetic algorithm is utilized to optimize the parameters of the fuzzy system, aiming to enhance profits and ensure accuracy in transactions at specific amounts. The experiments in this study involved selecting nine stocks and testing them against seven competing quantitative trading models. Upon comparing the profit rate, trading frequency, Sharpe ratio, and average return of each stock, eight stocks within the QF-FRL system achieved the highest returns and a greater number of transactions. Additionally, the QF-FRL system has also attained the highest average return and the second highest average Sharpe ratio. The results indicate that QF-FRL outperforms competing models, yielding higher profits and being particularly suitable for long-term investment. Moreover, it exhibits more favorable risk-adjusted returns and a notable degree of robustness.

Keywords Reinforcement learning · Twin delayed deep deterministic policy gradient · Adaptive-network-based fuzzy inference systems · Quantum finance · Quantum price level

1 Introduction

Stock trading represents a general investment in financial assets, accompanied by inherent risks and rewards. In stock trading, determining the best buying and selling strategies is crucial to maximize profits and mitigate investment risk. Identifying the optimal buying and selling points to maximize profits in a volatile stock market constitutes a long-term objective for investors. Fund management also plays a vital role in stock investment, enabling investors to control risks during the investment process based on stock performances, and implementing stop profit or loss strategies to exit markets at opportune times. Utilizing advanced quantitative trading methods can assist investors in

✉ Raymond S. T. Lee
raymondshtlee@uic.edu.cn

Chi Cheng
t330201602@mail.uic.edu.cn

Bingshen Chen
p930026134@mail.uic.edu.cn

[1] Guangdong Provincial Key Laboratory of Interdisciplinary Research and Application for Data Science, Beijing Normal University-Hong Kong Baptist University United International College, Zhuhai, Guangdong Province, China

determining the most effective buying and selling signals and in fund management.

Technical indicators and trading logics are commonly employed in traditional quantitative trading strategies. Technical indicators, like the moving average (MA) convergence divergence (MACD) and relative strength index (RSI), serve as references for identifying buy and sell positions in trading [15]. However, these technical indicators may yield misleading signals during periods of severe market volatility. The inherent lag in these technical indicators often prevents timely response to market changes.

Building upon traditional quantitative trading methods, new approaches and models have been developed to enhance trading logic and strategies. These methods and models have yielded significant progress. For instance, the autoregressive moving average (ARMA) model utilizes mathematical approaches to analyze historical time series and forecast in quantitative trading, whereas the mean regression model employs stock price deviations and the mean for selecting investment targets. While both models have garnered good returns, the complexity, nonlinearity, and dynamic nature of financial markets restrict their capacity for timely adjustments to market fluctuations. Trading based on reinforcement learning (RL) can adapt to evolving market conditions through interactive learning with the environment. This approach can be adjusted and optimized using real-time data and feedback signals, enhancing its adaptability to dynamic market changes. However, this model also presents certain limitations, including substantial data requirements, extended training periods, risk control challenges, and limited explanatory capabilities. In stock prediction and trading models, artificial intelligence technologies, including neural networks, genetic algorithms (GA), and fuzzy logic (FL), offer advantages in market modeling and prediction. Yet, they still necessitate investors' financial knowledge and experience, aspects often unaccounted for in automated trading. Concurrently, financial time series are laden with noise and market volatility, posing challenges for models in capturing recent changes and minimizing overgeneralization.

However, stock trading is frequently perceived as a zero-sum game in financial markets. Accurate forecasting in this domain is inherently complex and challenging. The chaotic nature of financial markets encompasses nonlinearity, uncertainty, and complexity. For instance, financial markets are impacted by a myriad of random factors, such as economic data, political events, and natural disasters. These random factors render market price fluctuations to be somewhat unpredictable and random. Price fluctuations in financial markets frequently deviate from linear relationships and exhibit complex nonlinear patterns. In this context, blind investment by investors may lead to an inability to optimize investment portfolios and transaction volumes, potentially resulting in additional transaction costs and uncertain returns.

In this paper, quantum finance is proposed to simulate the chaotic characteristics existing in financial markets. Quantum finance is the study of how to combine quantum mechanics and quantum field theory with contemporary artificial intelligence tools such as FL, GA, chaos, and fractals to model and explain financial predictions and quantum transactions on a global scale [16]. The combination of quantum finance and RL models will be a new way to solve the problem that the chaotic characteristics of the market reduce the generalization ability of the model.

This article proposes the QF-FRL system. It applies two agents, TD3 Trading Agent and quantum finance and an adaptive network-based fuzzy inference system (QF-ANFIS) Risk Control Agent, to obtain the optimal daily real-time trading strategy. Environmental observation factors include fundamentals and stock comprehensive technical indicators. DAEs are processed to make the input states easier to train. Then the output state is put into the TD3 model to train two preliminary action values. At the same time, the appropriate buying and selling points are identified based on the combination of actions and technical indicators, and then the actions are combined with the QPL indicator for fuzzy risk control to obtain the transaction amount. At the same time, GA is used to optimize the parameters in the fuzzy system. Nine stocks are then selected for comparisons.Experimental results show that the proposed QF-FRL system achieves instant determination of the best buying and selling strategy and obtains higher returns and stable profits.

The main contributions of this paper include the following:

1. Using DAE to process data enhances the generalization ability of the model. Build DDPG and TD3 model to determine the best buy and sell strategy based on action and MA.
2. Introducing QPL and ANFIS to implement risk control agency and obtain specific transaction amounts. GA is used to optimize fuzzy system parameters, achieving high and stable profits.
3. Making precise trades with specific transaction amounts, rather than buying and selling at full or half percent. The results obtained are representative.
4. Selecting nine stocks and testing them against seven other quantitative trading models and comparing the profit rate, trading times, Sharpe ratio, and average return of each stock that are aimed for comparative analysis.

The paper is organized as follows. Section 2 presents literature review on methods-based market trading.

Section 3 presents methodology of the proposed QF-FRL model. Section 4 describes model implementation details. Section 5 presents backtest experiments results and model evaluations. Section 6 presents conclusion with future work.

2 Literature Review

2.1 Traditional Quantitative Trading

Quantitative trading is a type of trading to make judgments and identify trading opportunities for enhanced profitability using mathematical models and analytics. Quantitative trading uses mathematical and statistical models to forecast market patterns. Based on the traditional quantitative trading, we can combine new methods and models, such as quantum finance, fuzzy learning, and RL. It provides simulated financial forecasts and financial markets, while increasing the ability of quantitative trading methods to handle the chaotic characteristics of the market.

Traditional quantitative trading models include the ARMA model proposed by Kirkpatrick and Dahlquist [15] and the generalized autoregressive conditional heteroskedasticity (GARCH) model proposed by Lo et al. (2000) who used mathematical and statistical methods to identify historical time series models for forecasting [19, 15]. The concept of uncertainty structure is particularly important when dealing with time series data that have inherent noise and uncertainty. In the context of ARMA and many other time series models, the "uncertainty structure" is primarily related to noise or error terms. Correctly addressing and modeling these uncertainties is critical to model accuracy and interpretability. Although these methods are based on financial theory, the high complexity, nonlinearity, and dynamic nature of financial markets limit their ability to make timely adjustments to market changes.

At present, it is rare to simply use traditional quantitative trading methods. Most quantitative trading methods incorporate some machine learning (ML) and deep learning (DL) methods. For example, de Oliveira et al. (2013) modeled high-frequency limit orders through support vector machines (SVM), showing effectiveness in real markets [25]. Wang et al. [32] applied deep attention networks (DANN) to model the interrelationships between assets to select portfolios from a stock pool [32]. Although many deep learning models have achieved financial prediction and algorithmic trading performance, the chaotic nature of the market reduces the generalization ability of the models. Regarding the treatment of chaotic properties, Cheng et al. (2014) explored the adaptive learning control of a class of switching strictly feedback nonlinear systems affected by external disturbances and input dead zones [7]. Li et al. [17] focused on using RL to solve complex control problems in chaotic systems [17]. This article will focus on the solution proposed by Lee [16]: quantum financial theory and quantum financial model [16].

2.2 Fuzzy System-based Trading

GA and FL are AI techniques used besides neural networks in stock forecasting and trading models. For example, Huang et al. [14] proposed biclustering algorithm and a fuzzy inference system to generate trading behaviors from a rule base to mine technical trading patterns [14]. Zhang et al. (2022) considered using sell orders to solve portfolio optimization problem by selecting optimal investment strategies based on a fuzzy risk value ratio and multi-objective GA [36]. Due to the accessibility of linear belonging functions, the returns of risky assets are regarded as triangular or trapezoidal fuzzy numbers in many fuzzy portfolio selection models such as Mashayekhi's integrated fuzzy multi-objective cross-efficiency model [21]. However, the linear membership function for historical data may be inapplicable. Yang et al. (2021) considered a generic fuzzy number called LR-power with a variable shape become favorable option for modeling fuzzy returns [33]. There are portfolio selection models such as Zhang et al. (2018) who proposed using fuzzy numbers and fuzzy set theory [35]. Gupta et al. (2021) proposed two multi-period models for portfolio optimization to accurately assess stock market prospects using coherent fuzzy numbers [12]. Gong et al. (2021) proposed a regret cross-efficiency evaluation model to measure portfolio efficiency with a regret theory-based fuzzy portfolio selection model to maximize the portfolio mean (return), skewness, and efficiency and minimize the variance (risk) on making behavioral decisions [11], while Tsaur et al. (2021) proposed a revised selection model to improve possible mean and variance values considering guaranteed return rate from overinvestment [30]. Chourmouziadis et al. (2020) proposed a new stock trading fuzzy system with technical indicators on medium-term prediction of optimal investment proportion [8]. They all embedded technical analysis into neuro-fuzzy systems.

These methods have advantages in market modeling and forecasting but still require investors' financial knowledge and experiences unrecognized by automated trading. Concurrently, financial time series contain noises and market volatility making it difficult for models to capture the latest changes and reduce generalization.

2.3 Reinforcement Learning-Based Trading

Reinforcement learning have algorithmic trading strategies characteristics. Chang et al. (2011) and Luo and Chen (2013) proposed stocks trading to make the best decision at each trading point through self-learning and updating strategy continuously to solve complex sequential decision problems with environment [6, 20]. There are advanced architectures and algorithms such as value-based critic networks and policy-based actor networks. These include Neuneier (1998) off-policy critic algorithm called *Q*-learning to allocate between index and cash [24], while Moody and Saffell (2001) proposed a trading system identifying investing policies without building a predictive model using actor-based optimized policies with recurrent RL algorithm instead value function [23]. Also, Dempster et al. (2006) proposed adaptive RL for foreign exchange market transactions [9]. Wang et al. [31] focused on pairs trading strategies to enhance performance using deep reinforcement learning (DRL) [31]. Carta et al. (2021) examined *Q*-learning agent's ensemble behavior in real stock markets [5]. Huang et al. (2021) combined adversarial learning with new sampling strategy to manage portfolios [13]. Azhikodan et al. (2019) used Neural Network model based on Deep Deterministic Policy Gradient and multi-reinforcement learning agent model validation used by Pendharkar and Cusatis (2018) to stock market forecast [1, 26].

Further, Bekiros developed an adaptive fuzzy model upon market changes' direction through an Actor-Critic agent based on FL and RL [3]. Skeepers et al. evaluated Deep Deterministic Policy Gradient, Ensemble of Identical Independent Evaluators, Proximal Policy Optimization and Fuzzy Deep Recurrent Neural Network (FDRNN) methodologies and extended FDRNN method for interaction with multi-asset markets [29].

Trading based on reinforcement learning is highly adaptable. It opens new trading areas by being able to proactively try different actions to learn and adjust based on feedback signals. Reinforcement learning can be optimized by means of long-term cumulative rewards, rather than just focusing on short-term gains and losses. This gives reinforcement learning an advantage in the pursuit of long-term stable returns. But trading based on reinforcement learning usually requires large amounts of data to train and learn. The model requires trial and error and learning through interaction with the environment, which can take a long time to achieve expected performance. Due to the trial-and-error nature of reinforcement learning, the model may encounter unprecedented extreme situations in actual transactions, resulting in the inability to effectively control risks.

To sum up, these three trading methods each have their own advantages and limitations. Combining quantum finance, FL, and reinforcement learning can help reduce constraints and achieve the goal of identifying optimal buying and selling points and money management.

3 Methodology

3.1 Denoising Autoencoder

Rumelhart (1986) introduced an autoencoder (AE) concept to process high-dimensional complex data facilitating neural networks developments [27].

AE attributes to unsupervised learning category where labeling training samples are not required.It uses an algorithm to relate data compression and decompression functions are Lossy to automatically learned from the samples. It is a type of neural network and a feed-forward networks special case to use one or more neural network layers directly in mapping input data to extract output vector as features.

AE model has a neural network structure consisting of three layers: input, middle, and output. Its purpose is to convert input x into x' by encoder and decoder to compare input x with output x' consolidated infinitely.

Currently, AE main usages are dimensionality reduction and denoising.

Traditional AE is generally used for data dimensionality reduction or feature learning, like Principal Component Analysis (PCA), but AE is flexible than PCA because it can characterize both linear and nonlinear transformations.

Bengio (2008) introduced denoising autoencoder (DAE) based on traditional AE and added random noise to input layer data to prevent overfitting problems [4]. It uses noisy decayed samples to reconstruct non-noisy clean input to a robust learned encoder and enhances model generalization. This strategy allows DAE to learn reflective input data intrinsic features. Since traditional AE merely relies on error minimization between input and reconstructed signals to obtain a middle-layer feature input representation, it can guarantee that data intrinsic features are extracted, but the reconstructed error may result features learned are only AE initial input copies. Therefore, a noise injection strategy motivated by DAE is introduced to avoid the problem as illustrated in Fig. 1. In this article, the activation functions used by DAE are sigmoid and ReLU. The structure consists of an input layer, a noise input layer, a code layer, and an output layer.

The noisy data x^* reaches to middle layer through noised input layer, and the signal turns into H, which can be represented by the following equation:

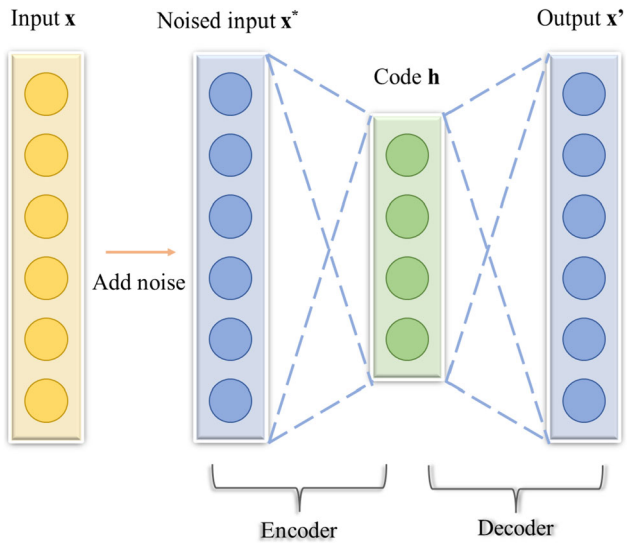

Fig. 1 Model of DAE

$$H = a(Wx^* + b). \tag{1}$$

In Eq. (1), a is the activation function, which can be a nonlinear function such as sigmoid and ReLU, W is the connection weight from the noised input layer to middle layer, and b is the bias of middle layer.

After signal H is decoded and delivered to output layer consisting of n neurons, it becomes x' as follows:

$$x' = a'(W'H + b') \tag{2}$$

In Eq. (2), W' is the connection weight of middle layer to output layer, and b' is the bias of output layer. Typically, the weight matrix W' is constrained to the transpose of weight matrix W, that is, $W' = W^{\mathrm{T}}$.

Then the network parameters are adjusted so that the final output x' is made as close as possible to match initial input x. There are many error calculations depending on the distribution assumed by input data; a typical squared error can be used by the equation below:

$$L(x, x') = \| x - x\prime \|^2 \tag{3}$$

This formula represents the square of the Euclidean distance. If the input data is a vector, cross-entropy method can be used by the equation below to minimize $L(x, x')$.

$$L_{\mathrm{H}}(x, x') = -\sum_{k=1}^{n} \ln[x_k \log x'_k + (1 - x_k)\log(1 - x'_k)] \tag{4}$$

Therefore, a DAE model is used to restore original data from noise-corrupted data and train with generalization capability for the proposed quantum finance and fuzzy reinforcement learning-based multi-agent trading system (QF-FRL).

3.2 Reinforcement Learning

3.2.1 Basic Concept

Reinforcement Learning (RL) is a part of machine learning (ML) motivated by behavior psychology to focus on how intelligence takes different actions in an environment to maximize cumulative reward.

There are two main subjects in RL: an agent and an environment. An agent learns and adjusts through continuous interaction with the unknown environment. At each time point t, an agent observes the environment to obtain state S_{f} and makes corresponding action A_{f} according to state S_{f} and forward R_{f}. The environment gives feedback forward R_{t+1} according to agent's action A_{f} and updates the environment by agent's influence to produce a new observation S_{t+1}. A model of RL is illustrated in Fig. 2.

The interaction between agent and environment generates a sequence of

$$S_0, A_0, R_1, S_1, A_1, R_2, \ldots \tag{5}$$

This sequence processing is called a sequential decision process. Markov assumptions are commonly used to solve this sequential decision process in RL. A Markov decision process (MDP) is a five-tuple $< S, A, P, R, \gamma >$, where S is the state space, A is the action space, P is the state transfer matrix, R is the reward function, and γ is the discount factor. In MDP, the next state is generated only to the current state, which is represented by

$$P[S_{t+1} \mid S_{\mathrm{f}}] = P[S_{t+1} \mid S_1, \ldots, S_{\mathrm{f}}] \tag{6}$$

This means that only the current state S_{f} is retained to transform the next state S_{t+1} and ignored historical information. For a particular state s and its next state s', their state transfer probabilities are defined as

$$P_{ss'} = P[S_{t+1} = s' \mid S_{\mathrm{f}} = s] \tag{7}$$

Suppose there are n states to select, then the state transfer matrix P is defined as

$$P = \begin{bmatrix} P_{11} & \ldots & P_{1n} \\ \ldots & & \ldots \\ P_{n1} & \ldots & P_{nn} \end{bmatrix}$$

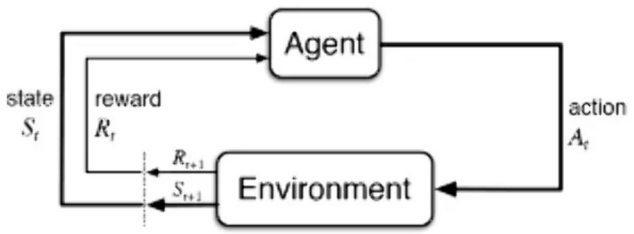

Fig. 2 Model of RL

The ith row of the matrix denotes that if the current state is S_i, then the probability of its next state being $1, \ldots, n$ is $P_{i1}, \ldots, P_{in}$, respectively. The probabilities in each row are sums to 1.

The reward equation represents reward expectation from state s to state s':

$$R_{\mathrm{s}} = E[R_{t+1} \mid S_{\mathrm{f}} = s] \tag{8}$$

The return for moment t depends on the reward sum from moment t_0 to t, denoted by U_{f}:

$$U_{\mathrm{f}} = R_{\mathrm{f}} + R_{t+1} + R_{t+2} + R_{t+3} + \cdots \tag{9}$$

Since state effect of current moment on state effect of future moment should be decreased, the decay is expressed in a discount factor and so the equation for return can be expressed as

$$R_{\mathrm{f}} = R_{t+1} + \gamma R_{t+2} + \cdots = \sum_{k=0}^{\infty} \gamma^k R_{l+k+1} \tag{10}$$

Emphasizing the basic concepts of reinforcement learning helps in the subsequent design of complex algorithms and models. A lot of subsequent work revolves around the design of reinforcement learning algorithms.

3.2.2 Actor-Critic-based Algorithm

An agent takes an action a in a different state s that is called a policy. To generate policies that maximize rewards, two main types of algorithms can be classified: value-based and policy-based.

Actor-Critic approach has advantages of both value-based and policy-based. In a policy gradient, the cumulative reward of an episode is like a critic, which determines an actor's learning direction so that the actor tends to learn logic with a higher cumulative Reward of the Critic. Therefore, the strategy gradient can be written as

$$g = E\left[\sum_{t=0}^{x_1} \psi_{\mathrm{f}} \nabla_\theta \log \pi_\theta(a_{\mathrm{f}} \mid s_{\mathrm{f}})\right] \tag{11}$$

where π here is the actor and ψ is the critic, which is a generalized framework.

The approach enables critic to learn the value function and determine how the strategy parameters of participants should be changed. In this case, actor raises calculating successive action advantage without optimizing the procedure of value function, while critic provides performance knowledge to actor. The actor-critic approach usually has good convergence compared to critic-only approach. A standard actor-critic model is illustrated in Fig. 3.

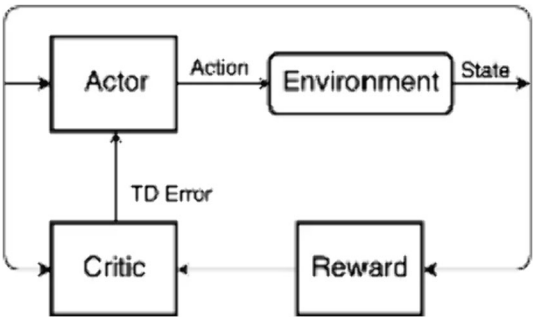

Fig. 3 Model of actor-critic

3.2.3 Twin Delayed Deep Deterministic Policy Gradient (TD3, Twin Delayed DDPG)

TD3 is an improved DDPG model, while DDPG is an Actor-Critic-based policy for continuous behavior learning method.

3.2.3.1 Deep Deterministic Policy Gradient (DDPG) Google Deepmind (Lillicrap et al. 2016) proposed DDPG as a policy learning approach that incorporates deep learning neural networks into DPG [18]. The key improvements relative to DPG which was proposed by Silver et al. (2014) are adopting convolutional neural network as a simulation of Q function and policy function μ, i.e., the Q network and policy network, using deep learning methods to train the neural network $a_{\mathrm{f}} \sim \pi_\theta(s_{\mathrm{f}} \mid \theta^\pi)$ [28]. The Q function implementation and training approach using Deepmind's DQN algorithm is the Q function method used by Alpha Go (Mnih et al. 2015) [22].

Deep Deterministic Policy Gradients (DDPG) can solve continuous control problems with high-dimensional state and action spaces in continuous space. It utilizes experience replay and deep neural networks to effectively handle high-dimensional state space problems. Because DDPG uses deep neural networks, training instability may occur. The DDPG algorithm uses random policy noise to explore the state-action space to achieve the purpose of controlling policy convergence. The uncertainty introduced by this noise cannot always be accurately simulated to the required exploration range, and may introduce unnecessary and meaningless actions, resulting in inefficient exploration.

3.2.3.2 Twin Delayed Deep Deterministic Policy Gradient (TD3) TD3 is an on-line off-policy DRL algorithm to solve sequential control issues improved from DDPG algorithm (Fujimoto et al. 2018) [10]. Essentially, TD3 algorithm integrates the ideas of double Q-Learning algorithm into DDPG algorithm.

TD3 can better handle continuous action space, and its performance is better than the DDPG algorithm. When

dealing with tasks in high-dimensional state space, TD3 has less training time and better performance than existing algorithms (such as DDPG). By introducing a delayed update mechanism and two target networks, the training process is made more stable, better strategies can be found adaptively, and better versatility can be achieved on different tasks.

3.3 Quantum Finance and Quantum Price Level

3.3.1 Quantum Finance Model

Lee [16] proposed Quantum Anharmonic Oscillator approach-based Quantum Price Level (QPL) [16]. The dynamics of financial instruments is modeled as Quantum Financial Particles (QFP) with wave-particle duality characteristics in Quantum Finance. The motions and dynamical significance of these quantum financial particles are affected by their intrinsic quantum energy field, the so-called quantum price field (QPF). Quantum Price Fields (QPFs) affect the motion and dynamics of these QFP composing QPL. From financial perspective, these QPLs correspond to Supports & Resistances.

Let Price Return r (Assuming they are performing anharmonic oscillation) perform as a particular QFP to analog the displacement from equilibrium position at time t, and the instantaneous return is given by

$$r(t,\Delta t)=\frac{p(t)-p(t-\Delta t)}{p(t-\Delta t)} \tag{12}$$

where $p(t)$ is the stock price at time t and Δt denote the time interval.

Time-Independent SE can be rewritten as

$$\left[-\frac{\hbar^2}{2m}\frac{\partial^2}{\partial^2 r}+V(r)\right]\varphi(r)=E\varphi(r) \tag{13}$$

The Hamiltonian operator $\widehat{H}$ is given by

$$\widehat{H}=\frac{\hbar}{2m}\frac{\partial}{\partial r^2}+V(r) \tag{14}$$

where $\widehat{H}$ contains kinetic and potential energy parts. $\hbar$ is the reduced Planck's constant representing the uncertainty of irrational transactions. m represents the intrinsic properties of the financial market. E is the eigen-energy and $\varphi(r)$ is the corresponding eigenfunction.

The return r can be described as wavefunction visualized by the probability density function (PDF):

$$\rho(r,t)=|\psi(r,t)|^2=|\varphi(r)|^2 \tag{15}$$

The dynamics of a financial model can be analyzed by key players such as Market Maker (MM), Arbitrageurs (ignored in analysis), Speculators (SP), Hedgers (HG), and Investors (IV) in financial markets. The key players are judged according to specific investment behavior. Define a financial asset's instantaneous demand and supply as Excess Demand z (Eq. 25); it can formulate instantaneous returns r(t), where F can be estimated approximately using a scaling factor γ representing the market depth for small Δz (Eq. 26).

$$\Delta z=z_+-z_- \tag{16}$$

$$\frac{\mathrm{d}p}{\mathrm{d}t}=r(t)=F(\Delta z)=\frac{\Delta z}{\gamma} \tag{17}$$

$$\frac{\mathrm{d}r}{\mathrm{d}t}=\frac{d^2p}{\mathrm{d}t^2}=\frac{1}{\gamma}\frac{\mathrm{d}(\Delta z)}{\mathrm{d}t} \tag{18}$$

Merge quantum dynamics of all significant financial market participants:

$$\frac{\mathrm{d}\Delta z}{\mathrm{d}t}=\left.\frac{\mathrm{d}\Delta z}{\mathrm{d}t}\right|_{\mathrm{MM}}+\left.\frac{\mathrm{d}\Delta z}{\mathrm{d}t}\right|_{SP}+\left.\frac{\mathrm{d}\Delta z}{\mathrm{d}t}\right|_{\mathrm{HG}}+\left.\frac{\mathrm{d}\Delta z}{\mathrm{d}t}\right|_{\mathrm{IV}} \tag{19}$$

$$\begin{aligned}\frac{\mathrm{d}\Delta z}{\mathrm{d}t}=&-\gamma\alpha_{\mathrm{MM}}r-\delta_{SP}r-\left(\delta_{\mathrm{HG}}-v_{\mathrm{HG}}r^2\right)r\\&+\left(\delta_{\mathrm{IV}}-v_{\mathrm{IV}}r^2\right)r\end{aligned} \tag{20}$$

where z_+ indicates long position, z_- indicates short position, and α is the market absorbability factors.

Combining with Eq. (18) can have

$$\frac{\mathrm{d}r}{\mathrm{d}t}=\gamma\frac{\mathrm{d}\Delta z}{\mathrm{d}t}=-\gamma\delta\mathrm{r}+\gamma v r^3 \tag{21}$$

where $\delta=\gamma\alpha_{\mathrm{MM}}+\delta_{SP}+\delta_{\mathrm{HG}}-\delta_{\mathrm{IV}}$ (damping term) and $v=v_{\mathrm{HG}}-v_{\mathrm{IV}}$ (volatility term).

Using Brownian price return in Langevin equation, the time independent quantum potential of the proposed QFFRL is given with overdamping case that $\frac{d^2r}{dt^2}=0$:

$$V(r)=\int\left(-\gamma\eta\delta r+\gamma\eta v r^3\right)dr=\frac{\gamma\eta\delta}{2}r^2-\frac{\gamma\eta v}{4}r^4 \tag{22}$$

where η is the market resistance coefficient.

Therefore, the Quantum Finance Time-Independent Schrödinger Equation (QFSE) of a Quantum Finance Particle can be written as

$$\left[\frac{-\hbar}{2m}\frac{d^2}{dr^2}+\left(\frac{\gamma\eta\delta}{2}r^2-\frac{\gamma\eta v}{4}r^4\right)\right]\varphi(r)=E\varphi(r) \tag{23}$$

QFSE can be combined with numerical computing techniques and finite difference method (FDM) to derive and evaluate the quantum price level (QPL).

3.3.2 Quantum Price Level Calculation

For numerical computation, QFSE can be solved using "λx^{2m} quantum anharmonic oscillators" method by Dasgupta's work with $m=2$. The equation is converted to

$$\frac{d^2\varphi_r}{dr^2} + \left(r^2 + \lambda r^4\right)\varphi_r = E\varphi_r \tag{24}$$

The energy levels can be approximated by polynomials:

$$\left(\frac{E(n)}{2n+1}\right)^3 - \left(\frac{E(n)}{2n+1}\right) = (K_0(n))^3\lambda \tag{25}$$

where $E(n)$ is the nth excited state energy of λx^{2m} AHO.

$K_0(n)$ are constants given by

$$K_0(n) = \left[\frac{1.1924 + 33.2383n + 56.2169n^2}{1 + 43.6196n}\right]^{\frac{1}{3}} \tag{26}$$

Then λ can be calculated using the Finite Difference Method (FDM). Since QFSE is symmetric to central-axis r_0 (ground state), the adjacent quantum dynamic for r_{+1} and r_{-1} can cancel out, so λ is given by

$$\lambda = \left|\frac{r_{-1}^2\varphi_{r_{-1}} - r_{+1}^2\varphi_{r_{+1}}}{r_{+1}^4\varphi_{r_{+1}} - r_{-1}^4\varphi_{r_{-1}}}\right| \tag{27}$$

Then the energy level can be calculated by Cardano's Method to solve the cubic equation:

$$E(n) = \sqrt[3]{-\frac{q}{2} + \sqrt{\frac{q^2}{4} + \frac{p^3}{27}}} + \sqrt[3]{-\frac{q}{2} - \sqrt{\frac{q^2}{4} + \frac{p^3}{27}}} \tag{28}$$

where $p = -(2n+1)^2$ and $q = -\lambda(2n+1)^3[K_0(n)]^3$.

This equation models financial particles in a financial market, where, like electrons in an atom, these particles remain in stable ground-state orbits without external financial stimulus. However, when significant financial events occur, these particles can jump to higher or lower energy levels, which are the QPLs, in discrete jumps. After such events, market makers absorb the stimulus and the financial particles return to equilibrium but at a new QPL.

Using quantum financial model and quantum price level can better simulate the complex financial market. The quantum price level will be used in subsequent fuzzy modeling. In the risk control agent, actions are combined with the QPL indicator for fuzzy risk control to obtain the transaction amount.

3.4 Neuron-Fuzzy System

Fuzzy logic and neural networks techniques are intermittently in engineering to answers unable provided by traditional methods. Neural networks incorporate computational learning features in fuzzy systems to receive interpretation and clarify system's representation. In general, neuro-fuzzy systems refers to all techniques based on neural networks and FL combined.

3.4.1 Fuzzy Systems

Fuzzy logic provides scientific and mathematical solutions for dynamic or chaotic problems. Zadeh (1965) proposed the Fuzzy Set Theory to describe an individual member of belonging degree with a real value instead of 0 and 1 [34].

The fuzzy inference mechanism generally consists of three stages. First, map each element into its fuzzy set according to a membership function (MF) to obtain fuzzy belongings as illustrated in Fig. 4. Second, process fuzzy rules based on firing strengths known as Fuzzification. Third, fuzzy values are transformed back into numerical values known as Defuzzification.

An effective fuzzy system can represent uncertainties with fuzzy variables to interpret results. It can adapt to interferences by extending the knowledge base of new rules. However, the system can only solve what is defined in its rule base depends on priori experiences. Uncertainties can be classified in a number of ways, depending on the circumstances. This article adopted fuzzy systems where the source of uncertainty is trading signals.

3.4.2 ANFIS Architecture

Neural networks have excellent learning, generalization capabilities and robust to disturbances. An Adaptive Network-based Fuzzy Inference System (ANFIS) architecture is used for QF-FRL model to implement a Takagi–Sugeno Fuzzy Inference System. ANFIS has four hidden layers as illustrated in Fig. 5. The first hidden layer maps each input into fuzzy sets according to membership functions to calculate the fuzzy belonging μ. The second hidden layer calculates the firing strength according to fuzzy belongings for the third layer to normalize. The fourth hidden layer determines the consequents f of rules. Then output layer evaluates the summation of $\overline{w}f$ as result. Learning algorithms are used into the system (ANFIS or QF-FRL) for parameters' modification to optimize performances. The fifth layer is a single superposition node that produces the final output, the defuzzification part of the fuzzy system, by superimposing all the inputs together.

Because ANFIS combines fuzzy system and neural network, the optimization efficiency and speed are improved, and the training speed is usually faster than that of traditional neural network. Thanks to FL, ANFIS architecture has good interpretability.

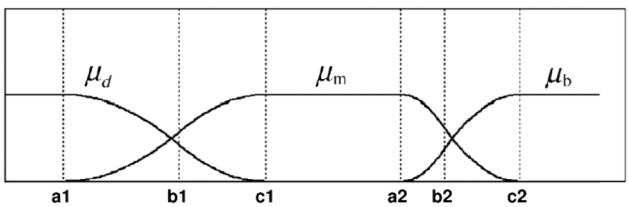

Fig. 4 Fuzzy membership function

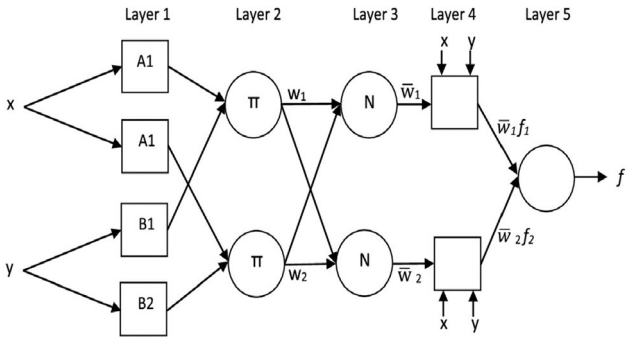

Fig. 5 Hybrid neuro-fuzzy control: ANFIS architecture

3.5 Genetic Algorithm

Genetic algorithm originates from biology concepts; it is "survival of the fittest" that governs natural selection according to Darwin's Evolution Theory. Chromosomes with high fitness values will generate more offspring than those with low fitness values, and the population will function better over successive generations.

The computation flow of a GA is illustrated in Fig. 6. In GA, the population of chromosomes is usually a set of solutions for a problem. This set can be encoded as a binary value with two types of genes 0 and 1.

In each generation, selection, crossover, and mutation operations are performed by individuals. For selection, the fitness of chromosomes will be evaluated according to a function $f(x)$. Higher fitness chromosomes will more likely be chosen in the following generation. For crossover operation, it is implemented by selecting a crossover point and exchanging genes at both ends of crossover point. For mutation, a single gene in a chromosome has the option to mutate into other types. In binary case, it can be implemented simply by reversing the digit. The halting criterion usually relies on the best fitness value of a chromosome in the population or the generation count.

GA has an advantage in finding the global optimal solution based on Evolution Theory. Its optimum outcome can be discovered independently of initial circumstances with mutational mechanism to strengthen its search capabilities. GA is easy to extend and use with another algorithm. Therefore, it is used as the learning algorithm in ANFIS for QF-FRL system.

4 Quantum Finance and Fuzzy Reinforcement Learning (QF-FRL)-Based Multi-agent Trading System

4.1 QF-FRL—System Framework

QF-FRL system consists of three parts: (1) extract stock features and processing, (2) use TD3 Agent for trading, and

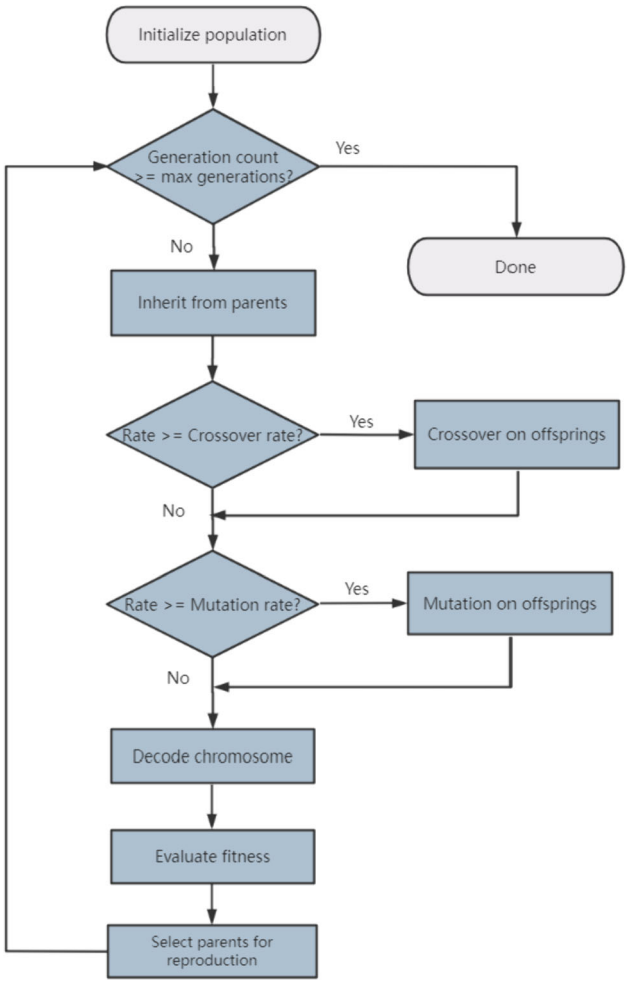

Fig. 6 Computation flow of genetic algorithm

(3) use QF-ANFIS Risk Control Agent for risk control to avoid excessive losses during the trading process. An overall structure of QF-FRL system is illustrated in Fig. 7.

First, the environment passes the processed dataset to Trading Agent as state, the Trading Agent will use the state to obtain the corresponding action value and pass the action to Risk Control Agent, and then an amount value, which is the transaction amount obtained by the Risk Control Agent, is reinstated to the environment. Then, the environment uses this amount to calculate profit and corresponding reward and passes the reward to the next state to Trading Agent again for the next action.

4.2 Denoise the Observations

After obtaining financial time series, noise reduction on observations is performed. The main process of pseudo-code is shown below:

DAE algorithm

Input environment observation o_t , and denoising required columns C
for each column i in C **do**
  Initialize an autoencoder model M with encoder-decoder parameters θ, θ'
  Initialize an Adam optimizer
  Sample o_t^i in the minibatch with a total of n_batches
  for b = 1, n_batches **do**
    Normalize b^{th} batch in o_t^i as x
    Generate random noise that satisfies the standard normal distribution
    x with the addition of noise becomes x^*
    for episode = 1, M **do**
      Using the encoder: h = f(x^*)
      Using the decoder: $x' = g(h)$
      Compute MSE loss of x^* and x'
      Back-propagate the gradients
    end for
    Store x' in s_t
  end for
end for
Get final denoised state representation s_t
Return s_t

Essentially, the algorithm is designed to train an autoencoder to denoise the data. It does this by deliberately introducing noise to the input data and then training an autoencoder to reconstruct the original noise-free data. The intuition behind this is that by doing this, the autoencoder learns to extract and preserve the most important features of the data while ignoring noise. In this algorithm, the batch size is 32 and the activation functions are sigmoid and ReLU. An example of DAE performance is illustrated in Fig. 8.

The DAE signal seems smoother than the ORIGIN signal. It oscillates around certain values, with some deviation, but does not show as many sharp peaks or changes as the ORIGIN signal. This means that DAE attempts to capture the main trend of the ORIGIN signal while filtering out some noise or short-term fluctuations.

4.3 TD3 Trading Agent

DDPG is used as trading agent in QF-FRL first but replaced by TD3 model which is a modified DDPG model since it can solve problems of minor action changes and insufficient policy obtained by DDPG. The reason why DDPG is used is first of all because it is necessary to start with a known, relatively simple model, and then gradually iterate and improve based on experience. This ensures the performance and stability of the model after switching to TD3.

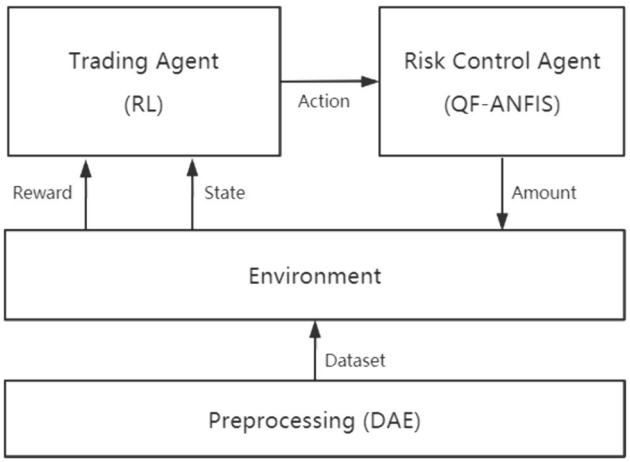

Fig. 7 System framework of the QF-FRL model

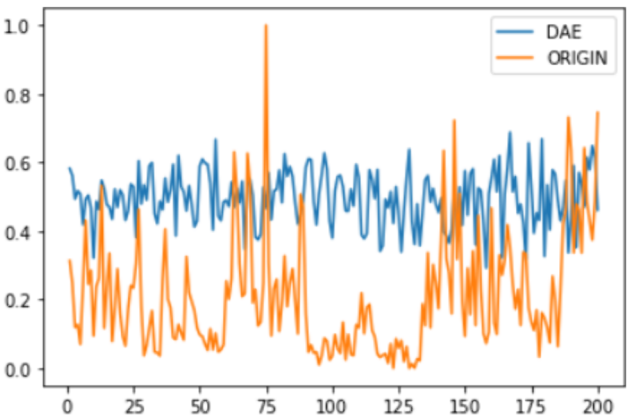

Fig. 8 Performance of DAE

4.3.1 DDPG Model in QF-FRL

A DDPG model consists of three main components: Critic network, Actor network, and Replay buffer. The overall structure of DDPG is illustrated in Fig. 9.

The Critic network is used to estimate the state-action value function (Q function) and provide a metric for evaluating the current strategy. The Critic network takes a state and an action as input and outputs a Q value representing the expected reward for performing a given action in a given state. The training goal of the Critic network is to minimize the temporal difference error (TD), which is the difference between the predicted value and the true value.

The Actor network is used to generate policies. An Actor network takes a state as input and outputs an action vector representing the action that should be taken in a given state. The training goal of the Actor network is to maximize the output (i.e., Q value) of the Critic network to improve the effectiveness of the strategy.

The Replay buffer is used to store experience data. At each time step, the DDPG model stores the current state, actions, rewards, next state, etc. into the replay buffer. The DDPG model will then randomly select a batch of experience data from the Replay buffer for training to improve training efficiency and stability.

The components of DDPG model are illustrated in Fig. 10.

Actor network first obtains a state from the environment and derives the current action to be stored in the replay buffer with current state, current reward, next state, and the current status of transaction, i.e., complete or incomplete. When the local network requires update, n random samples are selected from the replay buffer to learn and update the parameters of local network using optimizer. When local networks are all updated, they can update target networks.

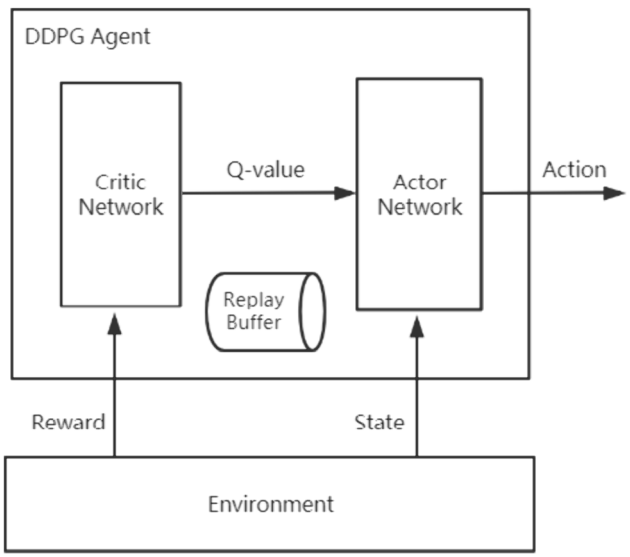

Fig. 9 Structure of DDPG model

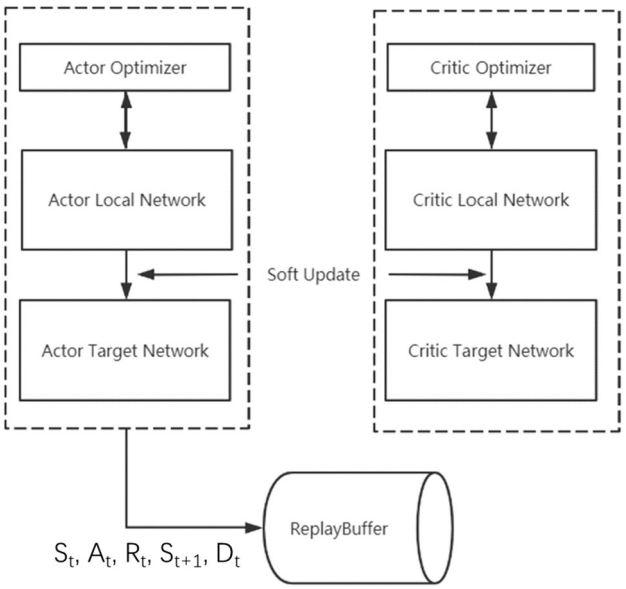

Fig. 10 Components of DDPG model

4.3.2 TD3 Model in QF-FRL

TD3 is Twin Delayed DDPG, which has been optimized and improved in three main aspects as listed in Table 1.

Accordingly, TD3 structure will change and update as illustrated in Fig. 11.

It is noted that there is one more local and target networks. First, the two target networks each derive a target Q and use the smallest value as the final target Q. Compare this target Q with two Q values derived from local networks to obtain two errors and use the sum of these two error values as the final error to update local network.

4.3.3 Trading in TD3

The trading logic used is real time. Each day two action values are obtained and multiplied by the open price of the current day to obtain buy and sell lines as illustrated in Fig. 12. These lines represent predefined price levels for executing sell and buy orders, respectively.

When the asset price falls and touches or falls below the "buy line," the buy order will be executed. This is based on the idea that an asset may be undervalued or declining, representing a potential buying opportunity to benefit from future price increases. When the asset price rises and touches or breaks the "sell line," the sell order will be executed. This indicates that the asset may be overvalued or at a peak, making it a good time to sell and realize a profit before any potential price drops. Additionally, transactions use 5 min of data, so 48 transactions can be made per day.

Table 1 TD3's main difference from DDPG

1	Clipped double_Q learning	Use two independent Critic networks to estimate the Q value; choose the smaller Q value to calculate the target Q, which can effectively alleviate the overestimation problem
2	"Delayed" policy updates	The Actor network uses delayed learning. The Critic network is updated more frequently than the Actor network in order to minimize the error before introducing the policy
3	Target Policy Smoothing Regularization	Noise is introduced into the Actor Target Network to enhance network robustness

However, it is noted that the sell line is difficult to reach during the trading process, which means that the stock was bought but unable to sell. Therefore, technical indicators such as MA indicator are combined to determine buying and selling points to solve this problem.

MA is used as a technical indicator to determine buying and selling points in this TD3 trading algorithm. MA is a common financial market technical analysis tool used to capture changes in price trends to help traders determine the best trading opportunities. The trading details in the pseudo-code are shown below:

TD3 trading algorithm

Initialize critic networks $Q_{\theta_1}, Q_{\theta_2}$, and actor network π_ϕ with random parameters θ_1, θ_2, ϕ
Initialize target networks $Q_{\theta_1'}, Q_{\theta_2'}$ and $\pi_{\phi'}$ with weights $\theta_1' \leftarrow \theta_1, \theta_2' \leftarrow \theta_2, \phi' \leftarrow \phi$
Initialize one replay buffer R and three Adam optimizers for each local network
for train_episode = 1, M **do**
  Initialize a random process $\mathcal{N}$ for action exploration
  Receive initial observation state s_1
  Run GA for 10 times to optimize parameters which are used for the fuzzy system
  for t = 1, T **do**
    Get current state s_t
    Select action with exploration noise: $a_t \sim \pi_\phi(s_t) + \epsilon, \epsilon \sim \mathcal{N}(0, \sigma)$
    Action a_t is used along with technical indicator MA to determine buy and sell points
    When meet buy or sell points, use the fuzzy neural network to determine the trade amount
    Use amount to calculate profit and reward r_t
    Get next state s_{t+1} and done situation d_t
    Store transition $(s_t, a_t, r_t, s_{t+1}, d_t)$ in R
    Sample a random minibatch of N transitions $(s_i, a_i, r_i, s_{i+1}, d_i)$ from R
    $\tilde{a} \leftarrow \pi_{\phi'}(s') + \epsilon, \epsilon \sim \text{clip}\,(\mathcal{N}(0, \tilde{\sigma}), -c, c)$
    Set $y_i = r_i + \gamma \min_{i=1,2} Q_{\theta_i'}(s', \tilde{a})$
    Update critic $\theta_i \leftarrow \text{argmin}_{\theta_i}\ N^{-1}\sum\left(y - Q_{\theta_i}(s,a)\right)^2$
    If t mod d **then**
      Update the actor policy using the sampled policy gradient:
      $$\nabla_\phi J(\phi) = N^{-1}\sum \nabla_a Q_{\theta_1}(s,a)\big|_{a=\pi_\phi(s)} \nabla_\phi \pi_\phi(s)$$
      Update the target networks:
      $$\theta_i' \leftarrow \tau\theta_i + (1-\tau)\theta_i'$$
      $$\phi' \leftarrow \tau\phi + (1-\tau)\phi'$$
    end if
  end for
end for
for t = 1, T **do**
  Get current state s_t
  Select action with exploration noise: $a_t \sim \pi_\phi(s_t) + \epsilon, \epsilon \sim \mathcal{N}(0, \sigma)$
  Execute action a_t, obtain reward r_t, next state s_{t+1} and done situation d_t
  Record profit and trading times
end for
Get the final profit and the final trading times

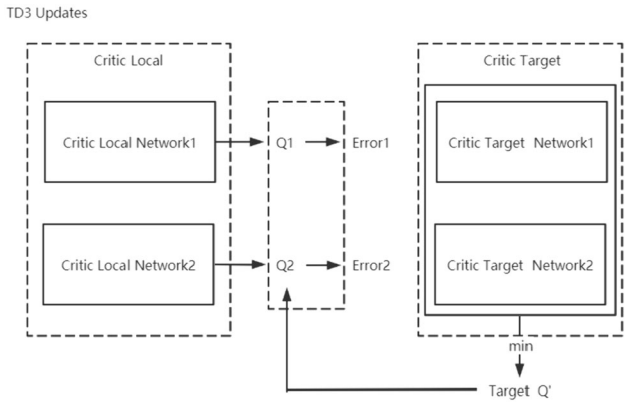

Fig. 11 Updated structure of TD3 model

Fig. 12 Trading logic

The TD3 trading algorithm combines traditional fintech indicators such as MAs with modern DRL technology. It employs a dual critic setting, explored action noise, and delayed policy updates (unlike TD3) to train agents that can make trading decisions in complex, noisy financial environments.The algorithm ultimately displays the total profit and the total number of trades executed.

4.4 QF-ANFIS Risk Control Agent

4.4.1 Fuzzy Inference Mechanism

A Risk Control Agent architecture is illustrated in Fig. 13. The computation flow follows a typical ANFIS with two inputs to real-time prices and one output representing the trading amount. The environment passes the processed dataset as a state to the transaction agent. Trading agents will use this status to obtain trading signals. Trading signals are real-time prices. After obtaining the instant price, fuzzy modeling and inference can be performed. The trading agent then transmits the trading signal to the risk control agent and obtains the amount value. This amount value is the transaction amount obtained by the risk control agent.

4.4.1.1 Fuzzy Identification and Categorization Here two fuzzy variables are defined: x_1 represents a trading signal according to the current price change compared with daily open price, and x_2 represents a trading signal according to current price value. For each fuzzy variable, three different fuzzy sets are divided into "BUY," "HOLD," and "SELL."

4.4.1.2 Fuzzy Modeling For fuzzy variable x_1, the membership function is defined in Fig. 14. The degree of belonging μ of each fuzzy set is calculated with 2 outputs of TD3 Trading Agent:α refers to a selling signal and β refers to a buying signal.

The numerical calculation of μ is given by

$$\begin{aligned} \text{mid} &= \alpha - (\alpha - \beta) \div 2 \\ \text{left} &= \text{mid} - (\alpha - \beta) \div 4 \\ \text{right} &= \text{mid} + (\alpha - \beta) \div 4 \end{aligned} \tag{29}$$

$$\mu_{\text{action}} \begin{cases} \mu_{\text{Buy}} \begin{cases} 1 & x \le \beta \\ \dfrac{\text{mid} - x}{\text{mid} - \beta} & \beta \le x < \text{mid} \end{cases} \\ \mu_{\text{Hold}} \begin{cases} \dfrac{x - \beta}{\text{left} - \beta} & \beta \le x < \text{left} \\ 1 & \text{left} \le x < \text{right} \\ \dfrac{\alpha - x}{\alpha - \text{right}} & \text{right} \le x < \alpha \end{cases} \\ \mu_{\text{Sell}} \begin{cases} \dfrac{x - \text{mid}}{\alpha - \text{right}} & \text{mid} \le x < \alpha \\ 1 & \alpha \le x \end{cases} \end{cases} \tag{30}$$

For fuzzy variable x_2, the membership function is defined in Fig. 15. The degree of belonging μ of each fuzzy set is calculated by QPL which performs as Support & Resistance lines in the market. For membership function, negative QPL_0 and positive QPL_0 are selected as reference lines. Quantum price oscillates generally within the ground state ($E0$). A selling signal or a buying signal is considered when x_2 approaches positive QPL_0 or negative QPL_0 accordingly.

The numerical calculation of μ is given by

$$\begin{aligned} b_1 &= \text{open} - \frac{\text{open} - NQPL_0}{2} \\ b_2 &= \text{open} + \frac{PQPL_0 - \text{open}}{2} \end{aligned} \tag{31}$$

$$\mu_{\text{price}} \begin{cases} \mu_{\text{Buy}} \begin{cases} 1 & x \le b_1 \\ \dfrac{\text{open} - x}{\text{open} - b_1} & b_1 \le x < \text{open} \end{cases} \\ \mu_{\text{Hold}} \begin{cases} \dfrac{x - b_1}{\text{open} - b_1} & b_1 \le x < \text{open} \\ \dfrac{b_2 - x}{b_2 - \text{open}} & \text{open} \le x < b_2 \end{cases} \\ \mu_{\text{Sell}} \begin{cases} \dfrac{x - \text{open}}{b_2 - \text{open}} & \text{open} \le x < b_2 \\ 1 & b_2 \le x \end{cases} \end{cases} \tag{32}$$

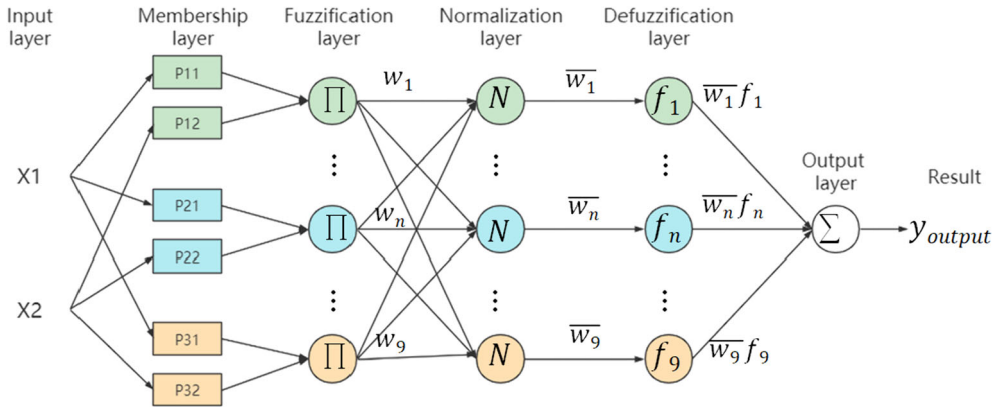

Fig. 13 QF-ANFIS architecture

Fuzzy membership function can be obtained by fuzzy modeling. The trading amount can be obtained by fuzzy reasoning combined with fuzzy membership function.

4.4.1.3 Takagi–Sugeno Model-Based Fuzzy Reasoning Based on previously defined fuzzy membership functions in Figs. 14 and 15, fuzzy relations are found by 9 alternative combinations. Takagi–Sugeno Model-Based Fuzzy Reasoning uses a polynomial function to represent each rule instead of indicating the rules directly. The ith rule can be represented as

$$\text{if } x_1 \text{ is } A_1^i \text{ and } x_2 \text{ is } A_2^i \ldots \text{ and } x_n \text{ is } A_n^i \text{ then } y^i = a_0^i + a_1^i x_1 + \cdots + a_n^i x_n$$

The weight of each rule is the multiplication of fuzzy set belonging degree for a fuzzy relation, which is given by

$$w^i = \mu_{A_1}^i(x_1) \times \mu_{A_2}^i(x_2) \tag{33}$$

The result x^* is calculated using the weighted average defuzzification method:

$$x^* = \frac{\sum_{i=1}^{k} w^i y^i}{\sum_{i=1}^{k} w^i} \tag{34}$$

where k indicates the total number of rules.

In output layer, x^* is put into the sigmoid function and reflected between 0 and 1. This value is used as the trading amount in back-testing for risk control. The optimal parameters or several better parameters can be selected by GA algorithm.

4.4.2 Genetic Algorithm-Based Optimization

GA is used in ANFIS training algorithm to learn the parameters in fuzzy rules. The optimization objective is to find a set of parameters (i.e., the "DNA" of an individual) that maximize the profitability of the stock trading strategy. This is done by calculating the fitness of each individual,

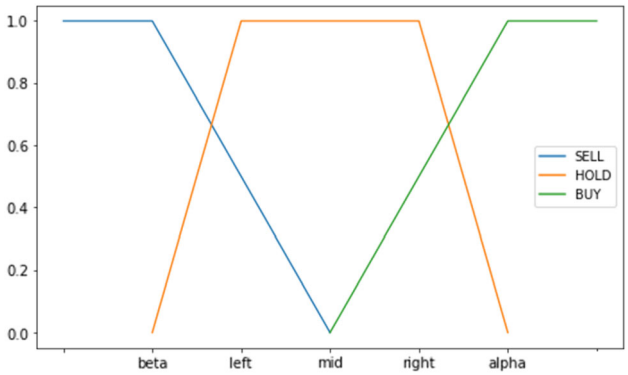

Fig. 14 Fuzzy membership function w.r.t price change

which is based on the profit of each individual. The higher the profit, the higher the fitness. The following parameters are set to constrain: DNA_SIZE = 5, POP_SIZE = 100, CROSSOVER_RATE = 0.8, MUTATION_RATE = 0.005, N_GENERATIONS = 10, PARA_BOUND = [− 1, 1].

In the population, the length of the DNA is 5(DNA_-SIZE, 5 by default)0. 100 chromosomes are initialized to represent 100 sets of parameters (POP_SIZE, 100 by default). The maximum number of generations (max generations) of the GA is set to 10 (N_GENERATIONS, 10 by default). PARA_BOUND is used to convert binary DNA into real parameters.

4.4.2.1 Chromosome Encoding and Decoding Since there are 2 dependent variables for each fuzzy rules in ANFIS, three constant parameters are available for each rule. A parameter can be coded into a binary string. Set the length of each parameter as D_L binary bits, the length of one chromosome C_L is given by

$$C_L = 27 \times D_L \tag{35}$$

To decode the chromosome, a limit bound for parameter reflection is defined. Let B_- denote a lower bound and B_+

denote a higher bound; two processes are operated to map the binary code into the range. First, transform the binary number into decimal system and, second, reflect the decimal number into the parameter range.

For a parameter code with a length of C_{L}, the decimal value d is calculated. The decoding parameter p is given by

$$p = \frac{d}{\left(2^{C_{\mathrm{L}}} - 1\right)} \tag{36}$$

The accuracy and precision of parameter relate to length C_{L} and the range $B_+ - B_-$, which are given by

$$\begin{aligned} \text{Accuracy} &= \frac{\left(2^{C_{\mathrm{L}}} - 2\right)}{\left(2^{C_{\mathrm{L}}} - 1\right)} \\ \text{Precision} &= \frac{\left(2^{C_{\mathrm{L}}} - 2\right)}{\left(2^{C_{\mathrm{L}}} - 1\right)} \times \left[\left(B_+ - B_-\right) + B_-\right] \end{aligned} \tag{37}$$

4.4.2.2 Fitness Level and Selection The fitness value of parameters can be simplified by determining trading profit *tp* in the training period. The fitness value of a chromosome can be calculated as the difference between the trading profit in optimum problem:

$$\text{Fitness} = tp - \text{minimum } tp \text{ in the population} + 1e^{-3} \tag{38}$$

where $1e^{-3}$ enlarge fitness to a positive number.

The sets of parameters are selected using roulette-wheel parent selection scheme. The chromosome with a higher fitness value has a higher probability to be selected as parents for the next generation. For a chromosome with a fitness value f, the probability selected is given by

$$p_i = \frac{f_i}{\sum_j^m f_j} \tag{39}$$

where f_i is the adaptation of the i individual, $\sum_j^m f_j$ is the sum of all individual finesses, and m is the population. Due to the parameterization of the GA (POP_SIZE, 100 by default), m is a constant in Eq. (39). The value of m is 100.

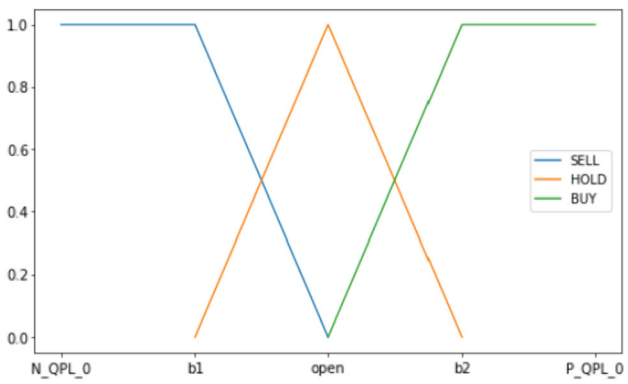

Fig. 15 Fuzzy Membership Function w.r.t Price

The selection process is based on the fitness of each individual in the population, which is directly related to profit generation in the context of stock trading. The fitness value is adjusted by a small constant ($1e^{-3}$) to ensure that it remains positive. The select function uses these fitness values to probabilistically select individuals for the next generation.

4.4.2.3 Crossover and Mutation After selecting the improved chromosomes, crossover and mutation are operated to create a better offspring. Crossover and mutation occurred accidently and is implemented by setting a crossover rate and a mutation rate.

For crossover operation, the offspring inherits part from an elder generation's chromosome before the crossover point and from another chromosome beyond the crossover point. Their binary genes are in alternate order to obtain a well-proportioned crossover of parameters. For mutation, a mutation point is selected randomly and the genes of either 0 or 1 are reversed.

Operations such as selection, crossover, mutation, and fitness evaluation are repeated to generate new populations until termination conditions are met. Through this method, the optimal parameters or several better parameters can be obtained.

For each individual (considered as a 'father'), there is a set probability (CROSSOVER_RATE, 0.8 by default) that crossover will occur. If crossover is to happen, another individual (a 'mother') is randomly selected from the population. A crossover point is randomly chosen, and the genes (binary values) from the mother are mixed into the father's DNA from that point onward, creating a child with a combination of both parents' genetic information. Each gene in the child's DNA has a small probability (MUTATION_RATE, 0.003 by default) of undergoing mutation.

5 System Implementation and Performance Analysis

5.1 System Implementation—Data

Stock data from the Shanghai Stock Exchange are utilized in the QF-FRL system, extracted via the Baostock library for each trading day. These data, including fundamental and technical indicator stock data, serve as the state input. Simultaneously, intra-day 5-min level real-time data are sourced for real-time trading. Baostock is a Python-based financial data interface library utilized to access stocks, indexes, and financial data of the China A-share market. It offers an easy-to-use API, facilitating developers in obtaining and processing financial data from official sources. The advantage of Baostock lies in its user-friendly API

Table 2 Data definitions

Item	Definition	Formula
Turn	Turnover rate	[Trading shares on the current day/total number of shares in circulation on the current day] × 100%
pctChg	Change percentage	[(Closing price of the current day − closing price of the previous day)/closing price of the previous day] × 100%
peTTM	Rolling price–earnings ratio	[Closing price on the current day/earnings per share TTM (Trailing Twelve Months) on the current day]
pbMRQ	Price–book ratio	[Closing price on the current day/net assets per share on the current day]
psTTM	Rolling price–sales ratio	[CLOSING price on the current day/sales per share on the current day]
pcfNcfTTM	Rolling price-to-current ratio	[Closing price on the current day/cash flow per share TTM (Trailing Twelve Months) on the current day]

interface and comprehensive financial data sources, enabling convenient access and analysis of China's A-share market data. It holds significant application value in the fields of quantitative investment, financial research, and data analysis [2]. For day-level data, fundamental data comprise both basic and company-specific information. Basic data encompass open price, highest price, lowest price, close price, trading volume, trading amount, turn, and pctChg. Company data consist of peTTM, pbMRQ, psTTM, and pcfNcfTTM. These data definitions can be found in Table 2. The technical indicator data comprise Bollinger High Band, Bollinger Low Band, MACD value, MA5, MA15, and MA30 values.

The dataset will undergo normalization, scaling each feature to a range between 0 and 1 using the MinMaxScaler from a preprocessing library. Subsequently, the dataset is inputted into the DAE for noise reduction processing. The output is then fed into the environment, serving as the state for the TD3 agent to determine the current day's action. The multiplication of the actions and the current day's open price are utilized as buying and selling reference points, alongside the MA15 indicator, to identify the final buy and sell points. When the day's real-time price triggers a buy or sell point, the action and QPL signals of the day are inputted into the QF-ANFIS Risk Control Agent. This considers both profit and risk to develop a balanced trading strategy for the day, followed by the use of a GA-optimized agent for profit maximization. In the TD3 algorithm, the following hyperparameters are configured: replay buffer size of int(1e4), minibatch size of 64, discount factor of 0.99, actor learning rate (and critic learning rate) of 1e−7, and 100 epochs. The fuzzy system incorporates two fuzzy variables, three fuzzy rules, and related membership functions: "BUY, " "HOLD," "SELL."

5.2 System Implementation—Experiment Settings

Seven models were selected for comparison, including BAH (Huang et al. 2014), PLR-WSVM [20], BIC-K-NN (Huang et al. 2014), MACD, MA, TD3, and BM-FM [14]. BAH (Buy and Hold) serves as a baseline model for comparing the performance of other trading strategies. PLR-WSVM combines piecewise linear representation (PLR) and SVM to predict stock trading signals in the financial investment field. BIC-K-NN (Bi-Clustering Algorithm and K-Nearest Neighbors) is an innovative biclustering algorithm utilized to identify effective technical trading patterns through a series of indicator combinations from historical financial data. MACD is a trend-following momentum indicator that illustrates the relationship between two MAs of a security's price. Meanwhile, MA is a commonly utilized technical indicator that smooths price data to form a single flowing line, facilitating easier identification of the trend's direction. TD3 represents the RL (reinforcement learning) model previously discussed. BM-FM is a fuzzy-related trading model that combines bicluster mining techniques to identify important trading patterns, construct a fuzzy rule base, and optimize a fuzzy inference system for predicting trading points. Compared with other models, the QF-FRL model integrates multiple algorithms and indicators, which helps to avoid the defects of using a single algorithm and a single indicator.

Table 3 Details of stocks for model comparison

Training period	2010/01/04–2010/11/18		
Testing period	2010/11/19–2011/08/18		
Downtrend			
Stock code	sh.600197	sh.600211	sh.600488
Steady trend			
Stock code	sh.600066	sh.600697	sh.600881
Uptrend			
Stock code	sh.600051	sh.600107	sh.600167

These nine stocks are selected and divided into downtrend, steady trend, and uptrend based on test data for model comparison. Further, the same training and testing

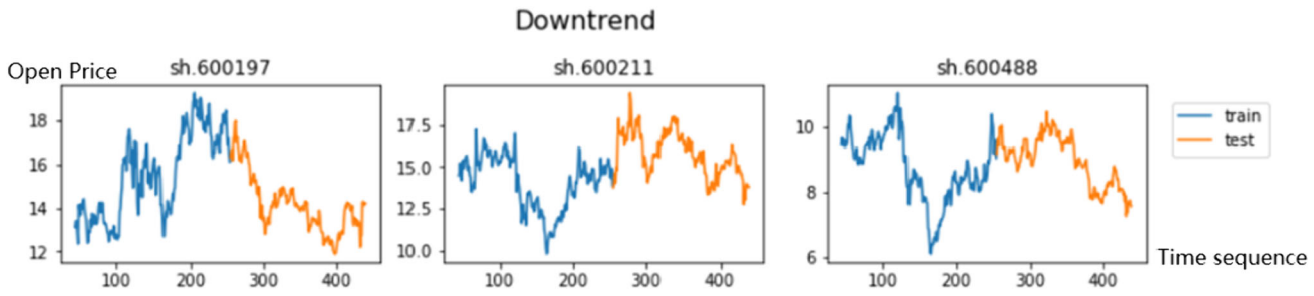

Fig. 16 Downtrend stocks' open price

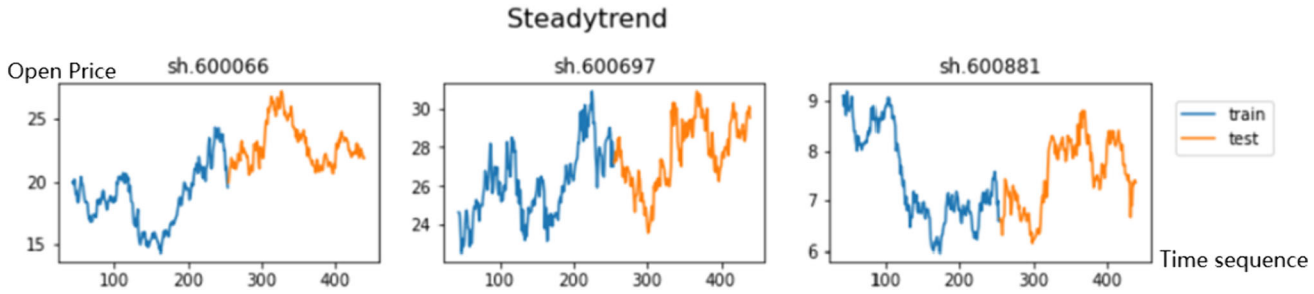

Fig. 17 Steady trend stocks' open price

periods determined by grid search with specific code and trend information for each stock are listed in Table 3.

The nine stocks are divided into three groups according to the trend. These three stock groups are illustrated in Figs. 16, 17, and 18. The 'train' data are typically used to build a model, and the 'test' data evaluate its performance. The model chosen here is BM-FM.

Through comparison, it is found that the trends of individual stocks in each group are moderate compared with the corresponding trends of the group. This shows that the grouping of these 9 stocks is reasonable and can be used for model comparison.

5.3 Performance Analysis—GA Optimization

GA is used as Risk Control Agent for optimization as illustrated in Fig. 19.

It showed that TD3 profit without risk control lies in the middle marked in blue, profit after 30 times GA marked in orange is lower than TD3 marked in blue, it can be interpreted that the number of optimization times is insufficient, and after increasing the optimization to 50 times marked in green, the profit is higher than TD3 which proves that GA optimization is useful and necessary.

When run multiple times, the GA will explore a larger search space. At the same time, the GA can avoid falling into the local optimal solution. Therefore, more rounds of GA optimization will lead to increased profits.

5.4 Performance Analysis—Trading Points

The results showing buy and sell points with corresponding trading volume of QF-FRL system in real-time trading are illustrated in Fig. 20. Green Triangles (Buy Points) indicate the points in time when buying occurred. They are positioned near relative lows of the price trend, which means buying was done when the stock price was lower, a strategy often referred to as "buying the dip." Red Inverted Triangles (Sell Points) indicate when selling occurred. Positioned near relative highs, which suggests that selling was done when the stock price was higher, a practice known as "selling the peak." The horizontal coordinates of triangles represent the time of trade and the vertical coordinates of the triangles represent real-time prices at the trade, while each triangle corresponds to a blue bar, which is the total trading volume of the trading day. Each blue bar corresponds to a triangle on the top chart. This means that on days when buying or selling actions occurred (represented by triangles), the trading volume is represented by the height of the corresponding bar. Taller bars indicate higher volume, meaning more stocks were traded that day. This chart effectively visualizes a trading strategy that aims to buy when prices are relatively low and sell when prices are relatively high. If executed correctly, this strategy can lead to profitable trades.

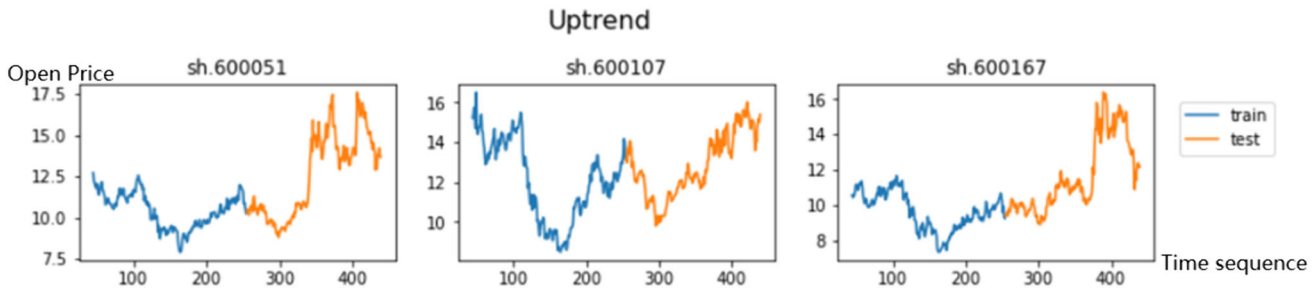

Fig. 18 Uptrend stocks' open price

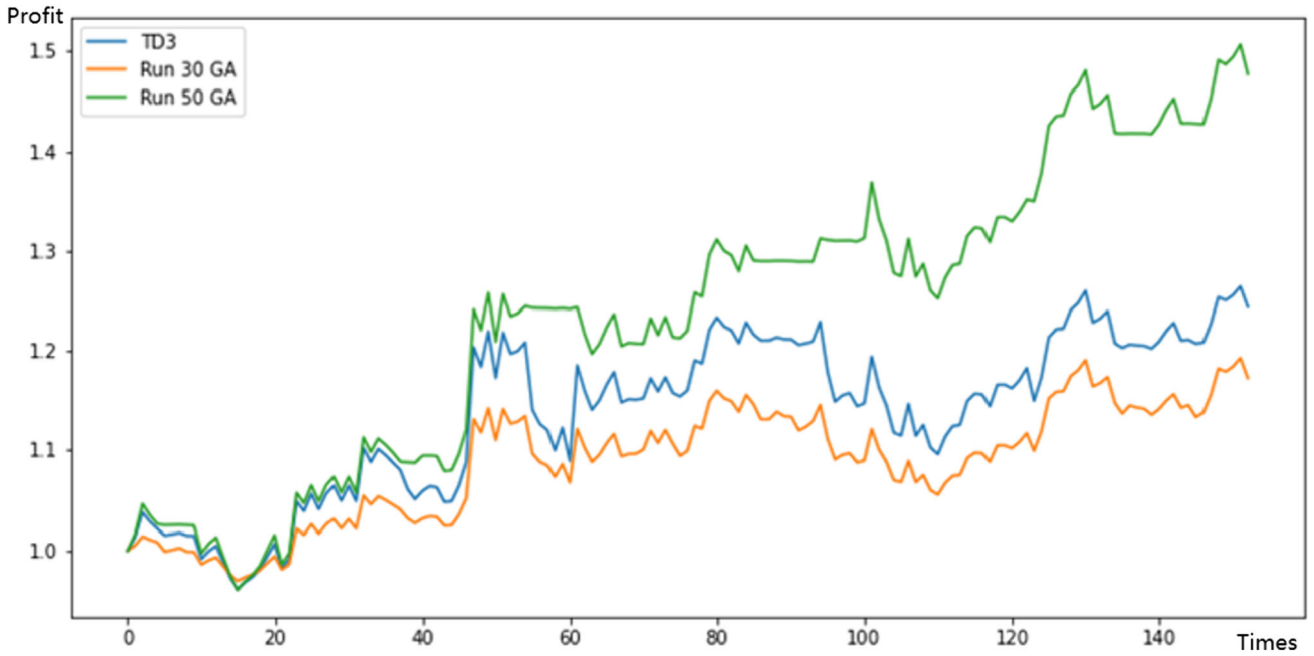

Fig. 19 Performance of GA

5.5 Performance Analysis—Experiment Results

The Sharpe Ratio is a measure used in finance to evaluate the performance of an investment compared to a risk-free asset, after adjusting for its risk. It is calculated by subtracting the risk-free rate (such as the return on U.S. Treasury bonds) from the investment's return and then dividing this result by the investment's standard deviation, which represents its volatility or risk. The formula for the Sharpe Ratio is

$$\text{Sharpe Ratio} = (R_{\text{p}} - R_{\text{f}})/\sigma_{\text{p}}, \tag{40}$$

where R_p is the return of the portfolio or investment. R_f is the risk-free rate. σ_p is the standard deviation of the portfolio's excess return. A higher Sharpe Ratio indicates a more favorable risk-adjusted return. A positive Sharpe ratio indicates that the stock has a positive return after deducting the risk-free rate. Assuming that the risk-free interest rate is 2%, use the return rate of each stock to represent its risk and return, and calculate the Sharpe ratio of each stock under different models and the average Sharpe ratio of each model.

Experiment results' comparison including profit ratio, trading times, Sharpe ratio, and average return of each method for nine stocks types are listed in Tables 4 and 5.

The highest profit rates for each stock are highlighted and bold. The results show that the 8 stocks in the QF-FRL system have the highest profits. Although the highest profit was not achieved in the last stock code 600167, it achieved the second highest profit, proving that the proposed system achieved satisfactory performance. Higher profits reflect the system's immediate determination of the best buying and selling strategy. The QF-FRL system requires a longer trading time, showing that it is suitable for long-term

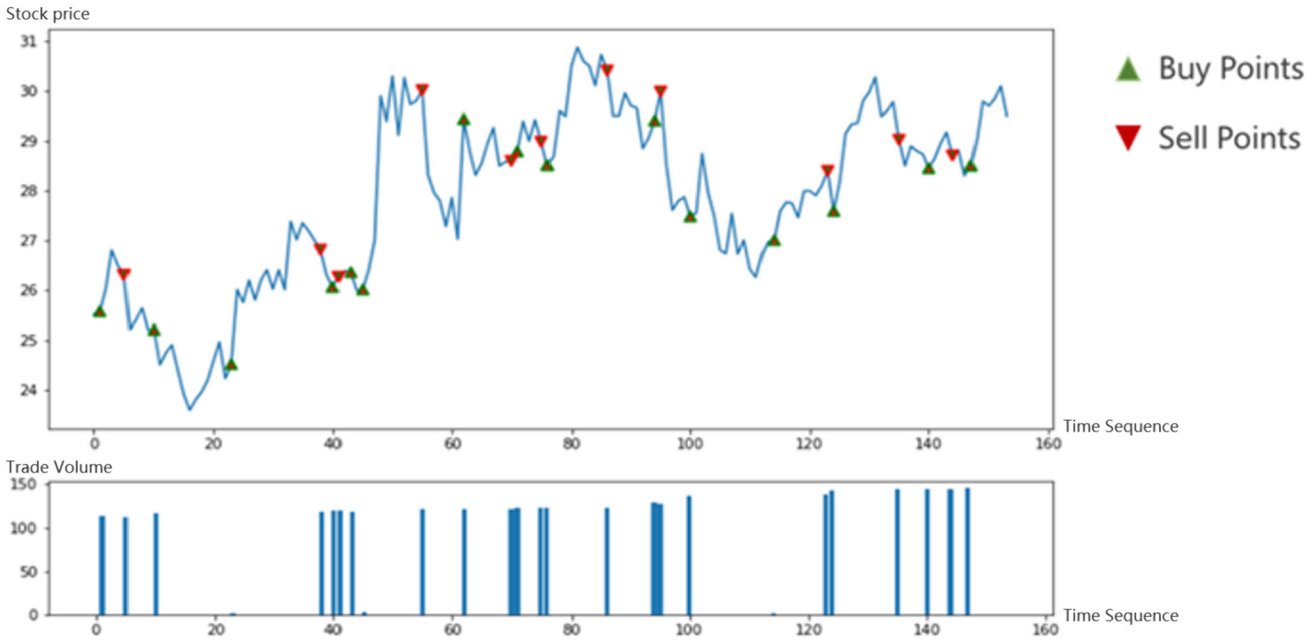

Fig. 20 Sample of trading

Table 4 Comparison result—part 1

Stock	BAH			PLR-WSVM			BIC-K-NN			MACD		
	Profit ratio (%)	Trading times	Sharpe ratio	Profit ratio (%)	Trading times	Sharpe ratio	Profit ratio (%)	Trading times	Sharpe ratio	Profit ratio (%)	Trading times	Sharpe ratio
600197	− 24.05	2	− 1.574	11.29	4	0.923	6.72	24	0.376	− 15.71	5	− 1.461
600211	− 13.2	2	− 0.918	5.38	18	0.336	20.07	24	1.441	− 8.67	6	− 0.880
600488	− 27.53	2	− 1.784	− 13.55	4	− 1.544	− 4.02	8	− 0.480	− 16.55	14	− 1.530
600066	− 9.26	2	− 0.680	− 3.88	4	− 0.584	13.39	22	0.908	− 8.92	8	− 0.901
600697	− 3.82	2	− 0.352	4.3	18	0.228	17.75	18	1.256	2.74	5	0.061
600881	1.9	2	− 0.006	10.72	14	0.866	21.51	32	1.556	− 5.07	6	− 0.583
600051	16.43	2	0.872	21.78	16	1.965	35.92	24	2.705	19.85	4	1.472
600107	16.27	2	0.862	15.33	13	1.324	10.63	20	0.688	13.92	5	0.983
600167	19.47	2	1.056	13.48	9	1.140	37.75	34	2.851	− 11.1	8	− 1.081
Average return (%)	2.64			7.21			17.75			− 3.28		
Average sharpe ratio (%)	− 0.28			0.52			1.26			− 0.44		

investment. The QF-FRL system achieved the highest average return (47.36%) and the second highest average Sharpe ratio (2.93%). In this system, the Sharpe ratio of each stock is positive, indicating that the QF-FRL system has more favorable risk-adjusted returns. For stocks with different trends, the QF-FRL system can also achieve good returns and more trading times, indicating that the model has a certain degree of robustness.

6 Conclusion

The article primarily focuses on the implementation of the QF-FRL system. The QF-FRL system employs two agents: TD3 Trading and QF-ANFIS risk control agent, to determine the optimal daily real-time trading strategy. Environmental observation factors encompass both fundamentals and a comprehensive range of stock technical

Table 5 Comparison result—part 2

Stock	MA			TD3			BM-FM			QF-FRL		
	Profit ratio (%)	Trading times	Sharpe ratio	Profit ratio (%)	Trading times	Sharpe ratio	Profit ratio (%)	Trading times	Sharpe ratio	Profit ratio (%)	Trading times	Sharpe ratio
600197	− 12.55	14	− 0.649	26.57	90	2.631	27.83	20	1.901	34.61	82	2.108
600211	0.23	11	− 0.079	25.88	85	2.557	26.4	24	1.796	40.69	77	2.502
600488	− 4.65	10	− 0.296	24.03	98	2.359	6.06	8	0.299	35.04	91	2.136
600066	− 8.88	14	− 0.485	21.31	70	2.068	15.36	18	0.983	25.34	123	1.509
600697	− 0.49	15	− 0.111	24.39	67	2.397	23.19	6	1.559	47.73	98	2.957
600881	− 7.76	16	− 0.435	37.38	66	3.788	38.01	32	2.650	50.6	110	3.142
600051	61.24	8	2.641	47.71	13	4.894	40.81	22	2.856	81.19	28	5.120
600107	19.71	9	0.790	45.59	13	4.667	23.32	20	1.569	58.74	35	3.669
600167	− 13.83	20	− 0.706	26.5	13	2.623	54.65	14	3.875	52.32	26	3.253
Average return (%)	3.67			31.04			28.40			47.36		
Average sharpe ratio (%)	0.172			3.11			1.70			2.93		

indicators. DAEs undergo processing to simplify the training of input states. Subsequently, the output state is fed into both the DDPG and TD3 models to train two preliminary action values. Simultaneously, appropriate buying and selling points are determined based on the combination of actions and technical indicators, followed by integrating the actions with the QPL indicator for fuzzy risk control. This process yields the transaction amount. Concurrently, a GA is utilized to optimize the parameters within the fuzzy system. Stocks and other quantitative trading models are then selected for comparison.

Experimental results indicate that the proposed QF-FRL system efficiently determines the best buying and selling strategy, yielding higher returns and stable profits. The QF-FRL system exhibits more favorable risk-adjusted returns and demonstrates a certain degree of robustness. However, the QF-FRL system also presents some disadvantages, as illustrated in Fig. 20. It is noteworthy that the low buying price sometimes only achieves the local minimum rather than the global minimum, and the selling price does not always attain the highest point. This suggests that although TD3 offers profitable buying and selling points, there is still room for improvement in its performance. For instance, running the GA a limited number of times may result in a local optimal solution, potentially diminishing the performance of the TD3 algorithm. Additionally, optimizing trading volume also represents a direction for enhancing profitability.

Therefore, in future work, the structure of QF-FRL needs to be optimized, including

(1) Utilizing the Softmax Deep Dual Deterministic Policy Gradient (SD3), an optimized version of TD3, for training neural networks; the softmax function and its gradient effectively propagate errors, thereby enhancing network training. In scenarios where the model exhibits uncertainty regarding certain categories, the softmax function can generate a more uniform probability distribution, aiding the model in acquiring additional information during training.
(2) Incorporating additional optimization methods, like neural networks, to fine-tune membership functions and rule parameters in risk control agents. Neural network-based optimization methods offer benefits including efficient, adaptive, nonlinear modeling, feature extraction, and scalability, positioning them as effective tools for managing a variety of complex tasks.
(3) Applying the QF-FRL system to other asset classes and set up more evaluation metrics. Develop more complex risk agents on top of existing systems.

Acknowledgements This paper was supported in part by the Guangdong Provincial Key Laboratory IRADS (2022B1212010006, R0400001-22), Key Laboratory for Artificial Intelligence and Multi-Model Data Processing of Department of Education of Guangdong Province and Guangdong Province F1 project grant on Curriculum Development and Teaching Enhancement on Quantum Finance course UICR0400050-21CTL.

Author Contributions Dr Raymond Lee: Supervision, project administration, funding acquisition, reviewing and editing. Chi Cheng: Conceptualization, methodology, reviewing and editing. Bingshen Chen: Conceptualization, methodology, writing—original draft preparation, formal analysis, data visualization, validation. Ziting Xiao: Investigation, literature review, software design, implementation.

Data Availability The datasets generated during and/or analyzed during the current study are available from the corresponding author on reasonable request.

Declarations

Conflict of interest The authors have not disclosed any competing interests.

Ethical Approval This article does not contain any studies conducted by any of the authors on human participants or animals.

Informed Consent Informed consent was obtained from all individual participants included in the study.

References

1. Azhikodan, A.R., Bhat, A.G.K., Jadhav, M.V.: Stock trading bot using deep reinforcement learning. In: Lecture Notes in Networks and Systems, pp. 41–49. Springer, Singapore (2019)
2. Baostock: http://baostock.com. Accessed 26 Feb 2024
3. Bekiros, S.D.: Heterogeneous trading strategies with adaptive fuzzy actor-critic reinforcement learning: a behavioral approach. J. Econ. Dyn. Control **34**(6), 1153–1170 (2010)
4. Bengio, Y., Vincent, P., Larochelle, H., Manzagol, P.A.: Extracting and composing robust features with denoising autoencoders. In: Machine Learning, Proceedings of the Twenty-Fifth International Conference (ICML 2008), Helsinki, Finland, 5–9 June 2008
5. Carta, S., Ferreira, A., Podda, A.S., Reforgiato Recupero, D., Sanna, A.: Multi-DQN: An ensemble of deep Q-learning agents for stock market forecasting. Expert Syst. Appl. **164**, 113820 (2021)
6. Chang, P.C., Liao, T.W., Lin, J.J., Fan, C.Y.: A dynamic threshold decision system for stock trading signal detection. Appl. Soft Comput. **11**(5), 3998–4010 (2011)
7. Cheng, Y., Xu, B., Lian, Z., Shi, Z., Shi, P.: Adaptive learning control of switched strict-feedback nonlinear systems with dead zone using NN and DOB. IEEE Trans. Neural Netw. Learn. Syst. **34**(5), 2503–2512 (2014)
8. Chourmouziadis, K., Chourmouziadou, D.K., Chatzoglou, P.D.: Embedding four medium-term technical indicators to an intelligent stock trading fuzzy system for predicting: a portfolio management approach. Comput. Econ. **57**(4), 1183–1216 (2020)
9. Dempster, M.A.H., Leemans, V.: An automated FX trading system using adaptive reinforcement learning. Expert Syst. Appl. **30**, 543–552 (2006)
10. Fujimoto, S., Hoof, H.V., Meger, D.: Addressing function approximation error in actor-critic methods. ICML **2018**, 1582–1591 (2018)
11. Gong, X., Yu, C., Min, L., Ge, Z.: Regret theory-based fuzzy multi-objective portfolio selection model involving DEA cross-efficiency and higher moments. Appl. Soft Comput. **100**, Article 106958 (2021)
12. Gupta, P., Mehlawat, M.K., Khan, A.Z.: Multi-period portfolio optimization using coherent fuzzy numbers in a credibilistic environment. Expert Syst. Appl. **167**, Article 114135 (2021)
13. Huang, S., Miao, Y., Hsiao, Y.: Novel deep reinforcement algorithm with adaptive sampling strategy for continuous portfolio optimization. IEEE Access **9**, 77371–77385 (2021)
14. Huang, Q., Yang, J., Feng, X., Liew, A.W.C., Li, X.: Automated trading point forecasting based on bicluster mining and fuzzy inference. IEEE Trans. Fuzzy Syst. **28**(2), 259–272 (2020)
15. Kirkpatrick, C.D., Dahlquist, J.: Technical Analysis: The Complete Resource for Financial Market Technicians, 2nd edn. Pearson, New Jersey (2011)
16. Lee, R.S.T.: Quantum Finance: Intelligent Forecast and Trading Systems. Springer, Singapore (2020)
17. Li, N., Li, X., Peng, J., Xu, Z.Q.: Stochastic linear quadratic optimal control problem: a reinforcement learning method. IEEE Trans. Autom. Control **67**, 5009–5016 (2020)
18. Lillicrap, T.P., Hunt, J.J., Pritzel, A., Heess, N., Erez, T., Tassa, Y., et al.: Continuous control with deep reinforcement learning. arXiV Preprint (2016). arXiv:1509.02971
19. Lo, A.W., Mamaysky, H., Wang, J.: Foundations of technical analysis: computational algorithms, statistical inference, and empirical implementation. J. Financ. **55**(4), 1705–1770 (2000)
20. Luo, L., Chen, X.: Integrating piecewise linear representation and weighted support vector machine for stock trading signal prediction. Appl. Soft Comput. **13**(2), 806–816 (2013)
21. Mashayekhi, Z., Omrani, H.: An integrated multi-objective Markowitz–DEA cross-efficiency model with fuzzy returns for portfolio selection problem. Appl. Soft Comput. **38**, 1–9 (2016)
22. Mnih, V., Kavukcuoglu, K., Silver, D., Rusu, A.A., Veness, J., Bellemare, M.G., et al.: Human-level control through deep reinforcement learning. Nature **518**(7540), 529–533 (2015)
23. Moody, J.E., Saffell, M.: Learning to trade via direct reinforcement. IEEE Trans. Neural Netw. **12**(4), 875–889 (2001)
24. Neuneier, R.: Enhancing Q-learning for optimal asset allocation. Adv. Neural. Inf. Process. Syst. **10**(1), 936–942 (1998)
25. de Oliveira, F.A., Nobre, C.N., Zárate, L.E.: Applying Artificial Neural Networks to prediction of stock price and improvement of the directional prediction index—case study of PETR4, Petrobras, Brazil. Expert Syst. Appl. **40**(18), 7596–7606 (2013)
26. Pendharkar, P., Cusatis, P.: Trading financial indices with reinforcement learning agents. Expert Syst. Appl. **103**, 1–13 (2018)
27. Rumelhart, D.E., Hinton, G.E., Williams, R.J.: Learning representations by back propagating errors. Nature **323**(6088), 533–536 (1986)
28. Silver, D., Lever, G., Heess, N., Degris, T., Riedmiller, M.: Deterministic policy gradient algorithms. In: International Conference on Machine Learning. PMLR (2014)
29. Skeepers, T., van Zyl, T.L., Paskaramoorthy, A.: MA-FDRNN: Multi-asset fuzzy deep recurrent neural network reinforcement learning for portfolio management. In: 2021 8th International Conference on Soft Computing & Machine Intelligence (ISCMI), pp. 32–37 (2021)
30. Tsaur, R.C., Chiu, C.L., Huang, Y.Y.: Guaranteed rate of return for excess investment in a fuzzy portfolio analysis. Int. J. Fuzzy Syst. **23**, 94–106 (2021)
31. Wang, C., Sandas, P., Beling, P.: Improving pairs trading strategies via reinforcement learning. In: 2021 International Conference on Applied Artificial Intelligence (ICAPAI) (2021)
32. Wang, J., Zhang, Y., Tang, K., Wu, J., Xiong, Z.: AlphaStock: a buying-winners-and-selling-losers investment strategy using interpretable deep reinforcement attention networks. CoRR (2019). arXiv:1908.02646
33. Yang, X.Y., Liu, W.L., Chen, S.D., Zhang, Y.: A multi-period fuzzy mean minimax risk portfolio model with investor's risk attitude. Soft. Comput. **25**, 2949–2963 (2021)

34. Zadeh, L.A.: Fuzzy sets. Inf. Control **8**(3), 338–353 (1965)
35. Zhang, Y.Y., Li, X., Guo, S.N.: Portfolio selection problems with Markowitz's mean-variance framework: a review of literature. Fuzzy Optim. Decis. Making **17**(2), 125–158 (2018)
36. Zhang, Y., Liu, W., Yang, X.: An automatic trading system for fuzzy portfolio optimization problem with sell orders. Expert Syst. Appl. **187**, 115822 (2022)

Chi Cheng is currently pursuing Master of Philosophy in Computer Science and Technology at the Beijing Normal University–Hong Kong Baptist University United International College (UIC). His research interests include Artificial Intelligence, Complex System, fintech, and natural language processing.

Bingshen Chen received her bachelor's degree in Computer Science and Technology at the Beijing Normal University–Hong Kong Baptist University United International College (UIC). Her research interests include quantum finance, neural network, chaotic neural network, and finance prediction.

Ziting Xiao received her bachelor's degree in Computer Science and Technology at the Beijing Normal University–Hong Kong Baptist University United International College (UIC). Her research interests include quantum finance, chaotic neural network, fuzzy neural networks, and finance engineering.

Raymond S. T. Lee received the B.Sc. degree in Physics from The University of Hong Kong, Hong Kong, in 1989, and the M.Sc. degree in Information Technology and the Ph.D. degree in Computer Science from The Hong Kong Polytechnic University (HKPolyU), Hong Kong, in 1997 and 2000, respectively. He is the Founder of Quantum Finance Forecast Center (QFFC) with more than 25 years of IT consultancy and R&D experiences in AI, chaotic neural networks, intelligent fintech system, and quantum finance. He is currently an Associate Professor with Beijing Normal University–Hong Kong Baptist University United International College working in the field of quantum finance, quantum anharmonic oscillators, chaotic neural oscillators, fuzzy-neuro financial systems, chaotic neural networks, and severe weather modeling and prediction.